

Kompilátory

1. Štruktúra a použitie kompilátorov, nástroje pre tvorbu kompilátorov.

- **Základné pojmy a štruktúra kompilátora (lexikálna analýza, syntaktická analýza, ..., rozdelenie na front-end a back-end).**
 - Frontend:
 - Lexikálna analýza
 - Rozdelenie na tokeny, funguje na regularných výrazoch zvyčajne.
 - Syntaktická analýza
 - Zoskupuje tokeny do stromu (stromu odvodenia / syntaktického stromu). Funguje na bezkontextových gramatikách.
 - Semantická analýza
 - Rieši veci mimo bezkontextových jazykov. Napríklad typové kontroly, alebo nedefinované premenne.
 - Generovanie medzikódu
 - Optimalizácia medzikódu
 - Backend:
 - Generovanie kódu
- **Vzťah k teórii jazykov. Syntax vs. sémantika.**
 - Lexery fungujú na regularných jazykoch
 - Parsery na bezkontextových
 - Robíme odvodenie
 - LL, LR gramatiky, CYK
 - Semantické kontroly typicky mimo CFG (ci sedia typy)
- **Kompilácia do iných jazykov, byte-kódu, strojového kódu. Porovnanie kompilátorov s interpretermi, JIT.**
 - Source-to-source-compilers
 - Kompilácia do iných vyšších jazykov
 - Cross-compilers (kompilujú pre inú platformu ako na ktorej bežia)
 - Stage-compilers:
 - Kompilácia do jazyka pre virtual machine (java bytecode)
 - Dekompilatory (z nižšieho do vyššieho jazyka)
 - JIT (just in time)
 - Kód sa interpretuje, ale niektoré časti sa pred volaním skompilujú (nie pred spustením, ale až počas runtimu). Vyhodnotí sa, že ktoré časti sa oplatí kompilovať (bude často volané).
 - Kompilatory vs. Interpretery:
 - Kompilovaný kód je rýchlejší, lebo je vopred preložený. Interpretovaný sa preklada po instrukciách počas behu.
 - Interpretovaný kód sa dá meniť počas behu
 - Pre beh interpretovaného kódu treba mať nainštalovaný interpreter.
- **Využitie kompilátorov pri statickej kontrole, formátovaní textu, spracovaní štruktúrovaných dokumentov.**

- TODO
- **Nástroje pre tvorbu kompilátorov (Flex, Bison alebo ANTLR)**
 - Flex:
 - Generuje lexery
 - Bison:
 - Generuje parsery, používa LALR parsovanie
 - ANTLR
 - Generuje aj lexery aj parsery, používa LL(*) parsovanie
- **Jednoduchý jednoprechodový kompilátor, rekurzívny zostup.**
 - TODO

2. Lexikálna analýza.

- **Dôvody oddelenia lexikálnej a syntaktickej analýzy. Token, Vzor, Lexéma.**
 - Rozdelenie na podulohy
 - Napr. whitespace v gramatike parsera by ho skomplikoval
 - Zvýšenie efektívnosti (specializácia lexera aj parsera)
 - Pri zmene kľúčových slov atď staci vymeniť lexer
 - Token:
 - Množina reťazcov so spoločným významom
 - Vystup lexera, vstup parsera
 - Pre syntaktickú analýzu je to terminal
 - Keywords, operatory, identifikatory, konstanty, ...
 - Vzor:
 - Pravidlo popisujúce množinu reťazcov pre daný token (typicky regex)
 - Lexéma:
 - Postupnosť znakov v zdrojaku, ktorá zodpovedá patternu pre nejaký token
- **Lexikálna analýza a odsadenie blokov ako syntaktická konštrukcia.**
 - Lexer číta vstup po blokoch do bufferu, lebo je to rýchlejšie ako po znaku a navyše sa tak ľahko robí lookahead (iba posun smerníka v bufferi)
 - Dvojbufferová schéma:
 - Keď pri čítaní lexemy prejdeme za polku, do druhej polky sa naplní vstup. Umožňuje to lepšie robiť lookahead.
 - Dajú sa použiť zárezky na koncoch polovic pre efektívnosť (netreba pozeráť, či už si na konci polky vždy).
 - Odsadenie blokov ako syntaktická konštrukcia:
 - Gramatika musí mať whitespacey
 - Je to považované za "zločin"
- **Regulárne jazyky, Nedeterministické konečné automaty, deterministické konečné automaty (DKA) a prechod od regulárnych jazykov k DKA.**
 - Tokeny tvoria regulárny jazyk zvyčajne
 - Spôsoby tvorby lexera:

- Prechodove diagramy (patterny sa špecifikujú pomocou diagramov, akoby automatov)
 - Každý diagram špecifikuje skupinu tokenov (rozne konečné stavy), napr. diagram pre relap
- Thomsonova metóda (patterny sa špecifikujú pomocou regexov)
 - Zostrojí sa NKA ku regexu (to je Thomsonova metóda)
 - Keď máme NKA pre s aj pre t, tak z nich ľahko pomocou epsilon-prechodov spravíme NKA pre st, alebo s*, alebo st
 - Simuluje sa NKA, alebo sa prerobí na DKA a ten sa simuluje
- Prechod od regulárnych jazykov k DKA:
 - Spraví sa napríklad regex, z neho NKA Thomsonovou metódou. Potom sa môže z toho spraviť DKA štandardnou konštrukciou z foje (stavy DKA sú množiny stavov NKA). Používa sa tam epsilon-closure.
- **Vyhľadávanie v texte (napr. Aho-Corasick).**
 - Spraví sa písmenkový strom z množiny patternov (po písmenkách)
 - To bude automat, tie patterny budú akceptačné stavy.
 - Pridajú sa fail hrany (väčšina asi pôjde do koreňa), ale inak klasika, že najdlhší prefix ktorý je suffix.
 - Pridajú sa ešte také hrany, že "kam sa da po fail hranách dostať"
- **Oprava chýb a zotavenie sa z chýb pri lexikálnej analýze.**
 - Lexer odhalí malú časť chýb (číta vstup, aplikuje regex a neda sa ďalej)
 - Opravy:
 - Zmazanie znakov zo vstupu
 - Pridanie znakov na vstup
 - Vymena zleho znaku za správny
 - Vymena susedných znakov

3. Syntaktická analýza zhora-nadol.

- **Bezkontextové gramatiky. Strom odvodenia. Abstraktný syntaktický strom.**
 - Strom odvodenia
 - Každý vnútorný uzol zodpovedá aplikácii pravidla
 - Nehovorí o tom, či je to ľavé/krajné/ine odvodenie
 - Ak existuje viac stromov odvodenia pre nejaké slovo, tak je gramatika nejednoznačná
 - Abstraktný syntaktický strom
 - Zjednodušený strom odvodenia
 - Operatory a kľúčové slová sú vnútornými vrcholmi
 - Vetvy odvodené cez chain rules sú zredukované
 - Využíva sa ako prechodná reprezentácia kompilovaného programu.
 - Prirodzenejšie reprezentuje štruktúru jazyka
- **Úprava gramatiky (eliminácia nejednoznačnosti a ľavej rekúzie, ľavá faktorizácia).**
 - Eliminácia nejednoznačnosti:

- Niekedy sa da eliminovat upravou gramatiky (napríklad G pre expressiony upravime tak, ze pridame termy a factory)
 - Nie vsetky CF gramatiky sa daju zjednotnacnit
 - Treba pri zhora-nadol, aj pri zdola-nahor
- Eliminacia lavej rekurzie:
 - Najskor sa eliminuje priama lava rekurzia
 - Pravidla $A \rightarrow Ax \mid Ay \mid b \mid c$ sa nahradia pravidlami $A \rightarrow bA' \mid cA'$, $A' \rightarrow xA' \mid yA' \mid \epsilon$
 - Potom sa usporiadaju neterminaly v nejakom poradí $A_1, \dots, A_n$
 - V tom poradí upravuj pravidla pre A_i vždy tak, aby na pravej strane bolo iba take A_j , že $j > i$. Cize ak je $k < i$ a mas pravidlo $A_i \rightarrow Akx$, tak to nahradis pravidlami $A_i \rightarrow D_1x \mid D_2x \mid \dots \mid D_mx$, kde D -cka su vsetky prave strany pravidiel pre A_k .
 - Nefunguje to pre gramatiky, kde sa da $A \Rightarrow^* A$, alebo $A \Rightarrow^* \epsilon$, ale to sa da tiež odstranit
 - Treba pri parsovani zhora-nadol
- Lava faktorizacia:
 - Ked je pravidlo $A \rightarrow xB_1 \mid xB_2$, tak sa to nahradi $A \rightarrow xC$, $C \rightarrow B_1 \mid B_2$, cize to rozhodovanie sa odklada na posledny mozny moment.
 - To suvisi s definiciou LL gramatik, ten prienik firstov
 - Treba pri parsovani zhora-nadol
- **Rekurzívny zostup, LL(1), LL(k).**
 - Rekurzívny zostup:
 - Vseobecna metoda, moze zahrnat backtracking, napríklad pri vybere pravidla.
 - Spravime proceduru pre kazdy neterminal. Na zaciatku procedury sa vždy vyberie pravidlo, ktorým dany neterminal prepiseme a zoberieme jeho pravu stranu, cez ktoru ideme. Ak najdeme terminal, tak ho matchneme so vstupom (alebo chyba). Ak neterminal X , tak zavolame proceduru $X()$. Chceli by sme sa zbavit backtrackingu pri tom vybere pravidla.
 - LL(1):
 - Prediktívne parsovanie je specialny pripad rekurzivneho zostupu, kde netreba backtracking. Na zaklade neterminalu a lookaheadu sa rozhoduje, ktore pravidlo pouzit. Daju sa tak robit LL(1) gramatiky.
 - Prechodove diagramy - konstruuju sa z gramatiky
 - Eliminujeme lavu rekurziu a spravime lavu faktorizáciu
 - Pre kazdy neterminal sa vytvori samostatny diagram.
 - Spravi sa pociatocny a koncovy stav a pre kazde pravidlo $A \rightarrow X_1 \dots X_n$ vytvorime cestu z pociatocneho do koncového stavu so sipkami oznacenymi $X_1, \dots, X_n$.
 - Pomocne funkcie FIRST a FOLLOW
 - FIRST(vetna_forma) je mnozina terminalov, ktorým moze zacinat slovo odvodené z vetna_forma

- FOLLOW(A) je množina terminalov, ktoré sa môžu vyskytnúť hneď napravo od A v nejakej vnútornej forme. Ak A môže byť posledný, tak aj \$ patrí do FOLLOW(A).
 - Pri výbere pravidla pre neterminal X sa vyberie pravidlo, ktoré má pravú stranu prava_strana, ak symbol na vstupe patrí do FIRST(prava_strana)
 - Parsovací tabuľka: $M[A, a]$. V nej sa hľadá pravidlo pre aktuálny neterminal a vstupný symbol.
 - Konštrukcia je z pravidiel $A \rightarrow \alpha$
 - Pre každý terminal x z FIRST(α) pridáme $A \rightarrow \alpha$ do $M[A, x]$
 - Ak epsilon patrí do FIRST(α), tak pridáme $A \rightarrow \alpha$ do $M[A, b]$ pre všetky b z FOLLOW(A).
 - Nedefinované záznamy sú chyby
 - Da sa zotaviť v móde paniky, synchronizačné tokeny sú pre neterminal A všetky terminály z FOLLOW(A).
 - Zotavenie na úrovni frázy sa dá tak, že prázdne políčka v parsovací tabuľke sa vyplnia smerníkmi na ošetrovacie chyby.
 - Gramatika je LL(1) práve vtedy, keď vieme zostrojiť parsovaciu tabuľku, čiže:
 - Pre každú dvojicu rôznych pravidiel $A \rightarrow \alpha \mid \beta$:
 - FIRST(α) $\cap$ FIRST(β) = $\{\}$
 - Ak epsilon patrí FIRST(β), potom FIRST(α) $\cap$ FOLLOW(A) = $\{\}$ a rovnako naopak
- LL(k):
 - Máme vyhlád k symbolov dopredu.
 - Platí, že LL(k) jazyky sú pod LL(k+1).
 - Parser sa rozhoduje podľa neterminalu a k symbolov, teda parsovací tabuľka je asi k+1 rozmerná
- **Všeobecné metódy pre bezkontextové jazyky: Earley-ho algoritmus, Cocke-Younger-Kasami.**
 - Earley:
 - Dostane gramatiku a vstupný string a vráti strom odvodenia (alebo množinu stromov odvodenia)
 - Ak má vstup n znakov, tak budeme mať stavy $S(k)$ pre $k = 0..n$
 - $S(k)$ obsahuje položky tvaru napríklad $(X \rightarrow ab.Ya, i)$, kde bodka označuje, kde sme v tom pravidle a i je, že na akej pozícii vstupu sme začali používať toto pravidlo (čiže v tomto prípade je to, že kde bolo to prvé a z pravej strany).
 - $S(0)$ obsahuje iba položku $(S \rightarrow .\alpha, 0)$, čiže s počiatočným pravidlom.
 - Opakuje 3 kroky, kým sa niečo mení:
 - prediction
 - Pre každý stav v $S(k)$, ktorý má bodku pred neterminalom pridaj do $S(k)$ položky pre ten neterminal, s pozíciou k
 - scanning

- Všetky položky z $S(k)$, ktoré majú za bodkou terminal, ktorý je prave na vstupe, pridaj s posunutou bodkou do $S(k+1)$
 - completion
 - Ak je v $S(k)$ nejaké pravidlo ($Y \rightarrow \alpha., j$), tak pozri do $S(j)$, tam budú možno nejaké pravidla, čo majú na pravej strane bodku pred Y . Tak take pravidla s bodkou posunutou za ten Y pridaj do $S(k)$.
 - Ak pociatočne pravidlo s bodkou na konci je v $S(n)$, tak slovo je akceptované.
 - CYK:
 - Máme slovo a gramatiku v prísom Chomského normálnom tvare a chceme zistiť, či sa to slovo dá odvodiť.
 - Dynamika. Budeme počítať tabuľku $M[i, j]$, kde na políčku $M[i, j]$ je množina neterminálov, z ktorých sa dá odvodiť podslovo začínajúce na pozícii i dlhé j znakov. Triválne je $M[i, 1]$ kvôli Chomského tvaru (" a " sa dá odvodiť iba z " $x_i a$ "). A potom ďalšie riadky z predchádzajúcich.
- **Oprava chýb a zotavenie sa z chýb pri syntaktickej analýze**
 - Zotavenie v mode paniky
 - Sú synchronizačné tokeny, pri chybe sa vynechá vstup po ďalší synchronizačný token (teda značná časť vstupu sa vynechá asi)
 - Zotavenie na úrovni frázy
 - Parser spraví lokálnu korekciu (napr. prefix zostavujúceho vstupu sa nahradí iným reťazcom)
 - Zotavenie pomocou chybových pravidiel
 - Do gramatiky sa pridajú pravidla pre generovanie chybových konštrukcií spolu s hlaseniami
 - Zotavenie pomocou globálnych korekcií
 - Hľadajú sa globálne najlepšie korekcie (čo najmenší počet atď)
 - Je to drahé
 - Strategie sa zvyknú kombinovať, typicky synchronizačné tokeny + na úrovni frázy

4. Syntaktická analýza zdola-nahor.

- **Bezkontextové gramatiky. Strom odvodenia. Abstraktný syntaktický strom.**
 - Rovnako ako v otázke 3.
- **Úprava gramatiky – eliminácia nejednoznačnosti.**
 - Rovnako ako v otázke 3.
- **Operátorovo precedenčné parsovanie.**
 - Operátorovo-precedenčné gramatiky sú podmnožinou LR gramatik
 - Da sa pre ne spraviť jednoducho shift-reduce parser
 - Prava strana pravidla nemôže byť epsilon
 - Neterminály na pravej strane nemôžu byť vedľa seba

- Vhodne napríklad na parsovanie aritmetických výrazov
- Precedenčné relácie:
 - $a > b$: znamená že a má väčšiu prioritu ako b
 - $a = b$
 - $a < b$
 - Slúžia na ohraničenie handle
 - $<$: Zčiatok handle
 - $>$: Koniec handle
- Medzi každé susedné terminály sa vloží $<$, $=$, alebo $>$
- Vždy sa najde prvé $>$, potom sa ide doľava cez všetky $=$ kým sa nenajde $<$ a potom sa zredukuje to medzi $<$ a $>$. Implementácia zásobníkom.
- Precedenčná tabuľka sa zvyčajne koduje do funkcií f , g , ktoré vracajú $f(a) < f(b)$, ak a má menšiu precedenciu ako b atď.
- **LR parsovanie, model LR parsera, LR parovací algoritmus.**
 - Budeme robiť redukciu vstupného reťazca a hľadať prave krajné odvodenie. Vždy teda chceme čo najľavejšiu časť zredukovať na neterminál. Tá najľavejšia zredukovateľná časť sa volá handle.
 - Životašopný prefix je prefix vnútornej formy, ktorý nepresahuje koniec handle.
 - Shift-reduce parser:
 - Číta vstup a má zásobník
 - Robí akcie:
 - Shift: Symbol zo vstupu sa pridá na zásobník
 - Reduce: Na vrchu zásobníka je handle, zredukuje ju
 - Error: Zavola funkciu na ošetrenie chyby
 - Accept: Úspešné ukončenie parsovania
 - Konflikty:
 - shift-reduce: Či už má zredukovať, alebo ešte čítať vstup
 - reduce-reduce: Podľa ktorého pravidla zredukovať?
 - Deterministický parser sa musí viesť rozhodnúť podľa zásobníka a k symbolov zo vstupu $\rightarrow$ LR(k) gramatiky
 - LR parsovanie:
 - TODO
 - Model LR parsera:
 - Číta vstup, má zásobník a ACTION a GOTO tabuľky, píše na výstup.
 - ACTION[s_m, a_i]: argumentami sú stav a čítaný symbol. Vráti niečo z:
 - shift s
 - reduce A $\rightarrow$ beta
 - accept
 - error
 - GOTO[s, A]:
 - Pomocná funkcia, dostane stav a neterminál a vráti nový stav. Tyka sa iba neterminálov, používa sa iba po redukcii.
 - shift akcie a goto tabuľka tvoria prechodovú funkciu DKA pre životašopné prefixy

- LR parsovaci algoritmus:
 - Nastavime pointer ip na zaciatok vstupu
 - While true:
 - Na vstupe je a, na vrchu zasobnika je stav s
 - if action[s, a] = shift s':
 - Vloz na zasobnik a, s'
 - ip++
 - else if action[s, a] = reduce A -> beta:
 - vyber 2|beta| symbolov zo zasobnika (znaky a stavy)
 - Nech s' je teraz na vrchu zasobnika
 - vloz A, potom goto[s', A] na zasobnik
 - vypis na vystup pravidlo A -> beta
 - else if action[s, a] = accept: return
 - else error
- **LR metódy – SLR, CLR a LALR.**
 - Pre kazdy z nich sa inac robi parsovacia tabulka (action a goto)
 - SLR:
 - Najjednoduchsi, ale najslabsi
 - LR(0) polozka:
 - [A -> ab.cYa], proste pravidlo s bodkou na pravej strane
 - Uplna polozka: bodku ma na pravej strane
 - Kontrukcia DKA:
 - Najskor spravme NKA. Rozsirime G o novy S' a pravidlo S' -> S.
 - Pociatocny stav je [S' -> .S]
 - Ak je v stave polozka [A -> a.Xb], tak pridame prechod na X do [A -> aX.b]
 - Ak je v stave polozka [A -> a.Xb], tak pridame prechody na epsilon do [X -> .alpha] pre vsetky X-pravidla.
 - Standardnou konstrukciou spravime DKA
 - Pomocna operacia closure
 - To je ze ked mame v stave polozku s . pred neterminalom, tak do toho stavu pridame aj polozky pre vsetky pravidla toho neterminalu s . na ziaciatku
 - Operacia goto na konstrukciu prechodovej funkcie
 - goto(I, X) je taka mnozina poloziek, ze v polozkach I posunieme bodku za X, kde sa da (ostatne zahodime) a z toho spravime closure
 - Operacia items na konstrukciu stavov DKA
 - Zacneme s C = {closure({[S' -> .S]})} a postupne pre vsetky prvky C a symboly X pridavame do C closure(goto(I, X)), ak tam este nie je
 - Kontrukcia SLR parsovacej tabulky:
 - Ak [A -> α.aβ] ∈ I_i a goto(I_i, a) = I_j, potom action[i, a] = shift j

- Ak $[A \rightarrow \alpha.] \in I_i$, potom $\text{action}[i, a] = \text{reduce } A \rightarrow \alpha$ pre všetky $a \in \text{FOLLOW}(A)$
 - Ak $[S' \rightarrow S.] \in I_i$, potom $\text{action}[i, \$] = \text{accept}$
 - Goto prechody su tak, ze ak $\text{goto}(I_i, A) = I_j$, tak $\text{goto}[i, A] = j$.
 - CLR:
 - Najsilnejši, ale nárôdnejši na implementáciu
 - V stave je uložená ďalšia informácia: symbol (terminal, alebo \$), ktorý môže nasledovať za pravou stranou pravidla. Dostaneme LR(1) položky.
 - V tej LR(1) položke ten lookahead je terminal, alebo \$, ktorý skutočne môže byť napravo od tej ľavej strany pravidla. Čiže je to nejaká podmnožina FOLLOW.
 - Zmení sa výpočet closure. Keď máme položku $[A \rightarrow \alpha.B\beta, a]$, tak pridáme do closure take položky $[B \rightarrow \cdot\gamma, b]$, že b patrí do $\text{FIRST}(\beta a)$
 - Goto sa nezmení, lebo nenaraba s lookaheadmi
 - Items sa nezmení, len pociatočná položka bude mať lookahead \$
 - LALR:
 - Kompromis medzi SLR a CLR
 - Počet stavov LALR parsera = počet stavov SLR parsera
 - Vychádza zo stavov CLR, ale tie sa zlučujú
 - Ak majú položky rovnaké pravidla, tak lookaheady sa zlučia a/b
 - Pri zlučovaní stavov môžu vzniknúť reduce/reduce konflikty
 - Na korektnom vstupe sa správa rovnako ako CLR, ale na nekorektnom môže spraviť ešte niekoľko redukcií kým vyhlási chybu
 - Vela riadkov v action tabuľke bude asi rovnakých, preto sa to optimalizuje smerníkmi do jednorozmerneho poľa, prípadne zoznamom akcií pre množiny stavov a symboly
- **Oprava chýb a zotavenie sa z chýb pri syntaktickej analýze.**
 - Chyba je detegovaná pri prázdnych zaznamoch v action tabuľke (nie v goto, lebo tam sa na prázdne políčka nikdy nedostaneme)
 - Zotavenie v mode paniky:
 - Vyberáme symboly zo zásobníka, kým nenajdeme stav, ktorý má prechod na vhodný neterminal A
 - Posuváme sa na vstupe, kým nenajdeme terminal, ktorý môže nasledovať za A
 - Pridáme na zásobník A, goto(s, A)
 - Zotavenie na úrovni frázy:
 - Ku každému prázdnemu zaznamu v action sa pridá funkcia na spracovanie chyby
 - Modifikuje symboly na zásobníku napríklad

5. Syntaxou riadené preklady.

Prepojenie syntaktickej analýzy s ďalšími fázami kompilácie. Može sa to robiť bez explicitnej konštrukcie stromu odvodenia, ale zväčša sa skonštruje a ten sa prechádza.

- **Atribútové gramatiky. Syntaxou riadená definícia.**
 - Atribútové gramatiky:
 - Je to syntaxou riadená definícia bez funkcií so side efektami
 - Syntaxou riadená definícia:
 - Vznikne rozšírením gramatiky o:
 - atribúty:
 - Su priradené ku symbolom gramatiky, nesú informácie, napríklad o type
 - semanticke pravidlá
 - Su priradené ku pravidlám. Slúžia na výpočet atribútov symbolov v rámci jedného pravidla.
 - Možu obsahovať funkcie so side efektami
 - Vrchol v strome odvodenia pozostáva zo symbolu gramatiky a poľa atribútov
 - Strom odvodenia s vypočítanými atribútami sa nazýva anotovaný strom odvodenia
 - Existuje prekladová schéma. To je iba detailnejšie ako SRD. Pravidlá sú povkladané medzi symboly na pravej strane pravidla.
- **Syntetizované a zdedené atribúty. S a L-atribútová definícia.**
 - Syntetizované: hodnoty sa počítajú z hodnôt detí
 - Dedicne: hodnoty sa počítajú z hodnôt rodičov a súrodencov. Čiže vnímajú kontext.
 - SRD s iba syntetizovanými atribútmi sa nazýva S-atribútová definícia
 - L-atribútová definícia môže obsahovať aj dedicne atribúty, ale sú tam pravidlá
 - Pre každé pravidlo $A \rightarrow X_1 X_2 \dots X_n$ musí platiť:
 - Dedicný atribút symbolu X_i na pravej strane je závislý iba na atribútoch symbolov $X_1, \dots, X_{i-1}$
 - Dedicný atribút X_i je závislý iba na dedicných atribútoch A
- **Vyhodnocovanie atribútov. Graf závislosti atribútov. Test na cirkularitu.**
 - S-atribútové definície sa dajú vyhodnotiť ľahko prechodom zdola nahor
 - Vyhodnocovanie atribútov L-atribútovej sa dá cez DFS. Najskôr deťom musíme poslať svoje dedicne. Potom vyhodnotiť deti a potom zbrať svoje zvršne.
 - Graf závislosti atribútov:
 - Vrcholmi sú atribúty
 - Hrana $a \rightarrow b$ je ak b závisí od a
 - Toto môžeme topologicky usporiadať a v tom poradí vyhodnocovať, ak to nemá cyklus
 - Test na cirkularitu:
 - TODO
- **Porovnanie metód zhora-nadol a zdola-nahor.**

- Pri LR parsovaní sa dajú pridať do zásobníku atribúty. Teda v zásobníku už nebudú iba stavy a symboly, ale v políčku zásobníka bude stav a hodnota atribútu. Pred redukciou sa použijú tieto atribúty na výpočet atribútu rodiča. To hovoríme o S-atribútových definíciách. Ale dá sa to rozšíriť aj pre L-atribútové.
- Pre L-atribútové definície sa dá parsovať iba keď máme LL(1) gramatiku bez ľavej rekurzie.
- Zhora nadol sa rozhodujeme iba podľa neterminálu a 1 symbolu zo vstupu. Zdá sa, že na začiatku vidíme celú frázu a životaschopný prefix, teda máme väčší kontext.

6. Podpora v čase behu (Run-time environments).

- **Organizácia a pridelenie pamäte (statické dáta, zásobník, dynamická pamäť).**
Zber smetí (garbage collection).
 - Statické dáta:
 - Priestor sa dá alokovať v čase kompilácie
 - Code area: inštrukcie, tam sa nemá zapisovať
 - Static area:
 - Pamäť vyhradená pre statické procedúry a statické dáta
 - Zásobník:
 - Dávajú sa tam informácie počas runtimu
 - Navratové adresy, parametre, informácie o scope
 - Bývajú tam aktívne zaznamy
 - Alokujú sa na stacku pri volaní
 - Lokálne dáta, parametre, miesto pre navratovú hodnotu, navratová adresa, info pre obnovenie (starý stack pointer)
 - Rôzne funkcie môžu mať rôzne veľké aktívne zaznamy
 - Počas kompilácie sa to dá zistiť (okrem funkcií s premenlivým počtom argumentov)
 - Dynamická pamäť:
 - Heap rastie oproti stacku
 - Garbage collection:
 - Keď je niečo v pamäti, ale nevieme sa k tomu už dostať, tak je to garbage
 - Zvyčajne máme iba šípku, že odkiaľ je pamäť voľná. Ale keď sa zaplní, treba upratať:
 - mark-scan:
 - Označí všetko, čo je živé v pamäti (prechádza grafom od premenných) a zvyšok sa môže zahodiť. Nakopíruje všetky živé veci na začiatok pamäte a zase má šípku, že "odtiaľto je pamäť voľná".
 - reference-count:
 - Počítame si referencie na veci, keď je 0 referencií, je to garbage

- Generational collectory su take, ze si objekty pocitaju, ze kolko upratovani uz prezili. A najskor sa vyhadzuju tie, co prezili malo upratovani.
- **Volanie funkcií a procedúr, odovzdávanie parametrov, lokálne a nelokálne premenné, aktivačný záznam, display.**
 - Odovzdavanie parametrov:
 - call-by-value:
 - Hodnota sa skopiruje
 - call-by-reference:
 - predava sa pointer
 - call-by-value-result (copy-restore):
 - Hodnota sa skopiruje a ta sa meni, ale na konci volanej funkcie sa vysledok ulozi do povodnej, cize sa zmeni na konci.
 - call-by-name:
 - Parameter sa nevyhodnoti, kym nie je potrebný
 - Volanie:
 - Volajuci:
 - Vytvori aktivacny zaznam na stacku
 - Vyhodnoti parametre a da ich do zaznamu
 - Ulozi navratovu adresu a dynamic link (aktualny stack pointer) a static link
 - Nastavi stack pointer na novy aktivacny zaznam
 - Posunie program counter do volanej funkcie
 - Volany:
 - Inicializuje lokalne data
 - Zacne vykonavanie
 - Navrat:
 - Volany:
 - Ulozi navratovu hodnotu do vyhradeneho miesta v aktivacnom zazname
 - Obnovi SP a PC
 - Volajuci:
 - Obnovi registre (do ktorych dal parametre)
 - Skopiruje navratovu hodnotu a pokracuje vykonavanie
 - Lokalne a nelokalne premenne:
 - Lokalna pre proceduru je taka, ze deklaracia sa nachadza v tej procedure
 - Nelokalna je ked nie je lokalna, ale deklaracia je v uzatvarajucom scope
 - Globalna je taka, ze deklaracia je v najvonkajsom scope
 - Static a dynamic link
 - Dynamic link:
 - Ukazuje na aktivacny zaznam volajuceho (cize stary SP)
 - Static link:
 - Ukazuje na aktivacny zaznam uzatvarajuceho scopu.

- Ak je volany deklarovaný vo mne a ja ho zavolám, tak jeho static link bude moj aktivacny zaznam.
 - Ak je volany deklarovaný k úrovni scopeu nado mnou, tak musím od svojho aktivacneho zaznamu prejsť po k static linkoch a to, kam sa dostanem, nastavím volanému ako static link
 - Keď chceme obsah premennej ktorá je v scope k úrovni nado mnou a na offsete off, tak použijeme (k, off), čiže prejdeme k static linkov a pozrieme na offset off.
- Display:
 - Pri static linkoch musíme niekedy traverzovať dlhý chain, kým sa dostaneme ku premennej, ktorú hľadáme (ak je veľký nesting depth)
 - Budeme udržiavať pole D take, že D[i] bude vždy ukazovať na posledný zaznam na stacku, ktorý je v nesting depth i.
 - Keď sa volá funkcia v nesting depth k, tak do jej aktivacneho zaznamu odložíme predchádzajúcu hodnotu D[k] a následne nastavíme D[k] na tento nový aktivacny zaznam. Pri ukončení funkcie sa stará hodnota D[k] obnoví.
 - Netreba teda už prechádzať chain, lebo hneď vieme najst posledný zaznam v nesting depth ktorú potrebujeme a tam sa pozrieme na offset.
- **Rozsah platnosti (scope) – statický (lexikálny) vs. dynamický.**
 - Statický (lexikálny):
 - Určuje sa podľa zdrojového kódu. Scope A je uzavretý v scope B, ak kód A-čka je vnútri kódu B-čka.
 - Dynamický:
 - Aktuálny call-tree určuje, ktorá deklarácia premennej sa použije
 - Každá premenná má stack hodnôt a vždy keď vznikne nový scope, tak tam vložíme novú hodnotu. To sa zo zdrojaku neda predpovedať že čo bude práve na vrchu stacku.

7. Generovanie medzijazyka.

- **Formy medzijazyka: Poľská bezzátvorková forma, štvorice (trojadresové inštrukcie) a trojice.**
 - Poľská bezzátvorková forma:
 - Poľská notácia:
 - Operator je prefixom
 - Ak majú všetky operatory rovnakú aritu, tak netreba zátvorky
 - Reverzná poľská notácia:
 - Operator je suffixom
 - Neumožňuje optimalizáciu
 - Vhodné iba pre malé zariadenia
 - Štvorice:
 - <operácia><operand 1><operand 2><výsledok>

- napríklad (+, a, b, tmp)
 - Trojice:
 - Tu sa používajú adresy trojíc na odkazovanie na ne:
 - 0: (+, a, b)
 - 1: (+, c, d)
 - 2: (*, 0, 1)
 - Keby sme do kódu trojíc vložili riadok, tak sa nám posúvajú adresy, čo asi nechceme, ale da sa to riešiť nepriamymi trojicami:
 - Tabuľka pre nepriamu adresáciu trojíc
 - Ale toto už nesetru miesto oproti stvoriciam
- **Generovanie medzijazyka syntaxou riadeným prekladom (atribúty kód a umiestnenie).**
 - Budeme generovať strom (resp. DAG ak memoizujeme mknode)
 - Post-order traverzovaním tohto stromu vieme dostať stvorice
 - Syntaxou riadený preklad vie mať ku pravidlám gramatiky priradené akcie na stavanie stromu (mknode, mkleaf, mkunode)
 - Každý výraz X bude mať atribúty:
 - code: Kód vygenerovaný pre X
 - place: meno premennej, v ktorej je uložená hodnota X
 - Napríklad: $E \rightarrow E_1 + E_2$ { $E.place := newtemp$; $E.code := E_1.code \parallel E_2.code \parallel gen(E.place := E_1.place + E_2.place)$ }
- **Popis základných konštrukcií: aritmetické a logické výrazy, príkaz priradenia, riadiace konštrukcie (if, while,...), volanie procedúr a funkcií.**
 - Aritmetické výrazy:
 - $E \rightarrow E_1 + E_2$ { $E.place := newtemp$; $gen(E.place := E_1.place + E_2.place)$ }
 - Priradenie:
 - $S \rightarrow id := E$ { $p := lookup(id.name)$; if $p \neq nil$ then $gen(p := E.place)$ else error }
 - if:
 - $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$ { $E.true := newlabel$; $E.false := newlabel$; $S_1.next := S.next$; $S_2.next := S.next$; $S.code := E.code \parallel gen(E.true := S_1.code \parallel gen("go to" S.next) \parallel gen(E.false := S_2.code)$ }
 - while:
 - $S \rightarrow \text{while } E \text{ do } S_1$ { $S.begin := newlabel$; $E.true := newlabel$; $E.false := S.next$; $S_1.next := S.next$; $S.code := gen(S.begin := E.code \parallel E.code \parallel gen(E.true := S_1.code \parallel gen("go to" S.begin))$ }
 - Volanie procedúr:
 - TODO z nových slajdov p7 treba
 - Ale zhruba:
 - $S \rightarrow \text{call } id \text{ Alist}$ { count = 0; save("returnaddress") }
 - $Alist \rightarrow E \text{ Alist}$ { save("param" E.place); count:=count + 1 }
 - $Alist \rightarrow e$ { gen("call" count id.ref) }
- **Spätné plátanie (backpatching).**

- Najjednoduchšie je spraviť 2 prechody (vygenerovať AST prvým a potom generovať kód pomocou post-order DFS)
- Ale ak chceme iba 1 prechod:
 - Problem su dopredne skoky
 - Nechame cieľ skoku prázdny a poznamenáme si ich do pola
 - Doplníme ich až keď sú príslušné adresy navštívi vygenerované
 - Každý prvok bude mať truelist a falselist
 - B.truelist je zoznam miest, kam treba doplniť label (vsade ten istý), ak B bude true
 - B -> true { B.truelist = makelist(nextinstr); gen("goto _"); }
 - To _ sa potom doplní neskôr (backpatchne)
 - B -> B1 || M B2
 - Ak B1 je true, tak na tú druhú časť netreba ísť a môže sa skocit rovno ďalej. Ak B1 bude false, tak ideme do B2 (čiže skácame na M). Preto to celé bude takto:
 - { backpatch(B1.falselist, M.instr); B.truelist = merge(B1.truelist; B2.truelist); B.falselist = B2.falselist; }
 - Napríklad B -> !B by iba swaplo B.truelist a B.falselist

8. Generovanie kódu.

- Cieľový počítač: CISC, RISC. Dĺžka a ceny inštrukcií. Zohľadňovanie cache a pipeline pri preklade. Špecifiká cieľových inštrukčných sád, ktoré treba zohľadňovať.
 - RISC:
 - 3-adresové inštrukcie
 - Veľa registrov
 - Málo adresných modov
 - Konštantne dlhé inštrukcie
 - Da sa pipelining, lebo všetko trvá rovnako dlho (okrem load a store)
 - CISC:
 - 2-adresové inštrukcie
 - Menej registrov
 - Viac adresných modov
 - Rôzne dlhé inštrukcie
 - Pipeline:
 - Hazards:
 - Strukturálne konflikty:
 - Ochránenie na súčasné použitie zdrojov počítača
 - Da sa riešiť zaradením prázdnej inštrukcie, alebo zdvojením hardveru
 - Datové konflikty:
 - Treba výsledok ešte nedokončenej inštrukcie

- Da sa riesit zaradenim prazdnej instrukcie
- Konflikty riadenia:
 - Este nepozname vysledok podmienky, ale uz mame mat v pipeline dalsiu instrukciu (ktoru z tych 2?)
 - Treba potencialne zrusenie casti vypoctu a zacatie odznova
 - Da sa riesit heuristikami (branch prediction)
- **Prideľovanie registrov. Prideľovanie registrov farbením grafu (register inference graph).**
 - Register allocation:
 - Pre kazdy bod programu urcit, ktore premenne vtedy budu v registroch
 - Register assignment:
 - Pre kazdy bod programu urcit konkretne priradenie registrov premennym
 - NP-tazke
 - Datove struktury pre pracu s registrami:
 - Register descriptor (ze aka premenna je v ktorom registri)
 - Address descriptor (ze kde vsade je nejaka premenna ulozena, cize registre, pamat, stack, heap, ...)
 - Toto sa da dobre udrzovat pri tej premennej v tabulke symbolov
 - Pocas generovania kodu udrzujeme tieto struktury
 - Generovanie kodu:
 - Algoritmus:
 - Ideme cez vsetky instrukcia[i] bloku od leadera
 - $L = \text{GetReg}(\text{instrukcia}[i])$
 - $\text{CodeGen}(\text{instruction}[i])$
 - $\text{Update}(\text{register_descriptor}, \text{address_descriptor})$
 - $\text{GetReg}(x = y \text{ op } z)$:
 - Ak y je v registri, tak vrat ten register
 - Ak nie je, tak vrat register kde je mrtva premenna
 - Ak nie je, tak prazdny.
 - Ak nie je, ale x je v registri, tak vrat ten register
 - Inac najdi nejaky vhodny odkial odlozis premennu do pamate, ak treba (podla address descriptoru tej premennej, ktora tam je ulozena)
 - Rovnaky postup pre z
 - Podobne aj pre x potom
 - Register inference graph (RIG):
 - Vrcholy su virtualne registre (teda ich moze byt viac ako skutocnych - temporaries)
 - Hrana medzi dvoma je, ked premenne v oboch uzloch su sucasne zive
 - Chromaticke cislo urcuje minimalny potrebny pocet skutocnych registrov bez spillovania
 - Farbenie je NP-tazke, ale my nepotrebujeme uplne optimalne
 - Heuristika:

- Odstranujeme vždy vrchol najmenšieho stupňa
 - Naspäť pripajame a dofarbujeme
- **Administrácia pamäte. Statická pamäť, zásobník, dynamická pamäť – heap.**
 - Každá procedura má:
 - Oblasť kódu
 - Oblasť statických dát
 - Oblasť dát na zásobníku
 - Heap sa explicitne alokuje a dealokuje (ale typicky je samostatný pre každý proces)
 - Volajúci musí alokovať priestor na zásobníku atď
 - Volany musí vykonať prácu a upratať zásobník
- **Rozdelenie programu na bloky, generovanie kódu z dagu.**
 - Blok:
 - Postupnosť príkazov, ktorá sa vždy vykona celá
 - Skladá sa z hlavičky (leader) a ostatných príkazov
 - Algoritmus rozdelenia znamená určiť hlavičky
 - Prvý príkaz programu je hlavička
 - Každý cieľ skoku je hlavička
 - Príkaz bezprostredne za príkazom skoku je hlavička
 - Iné hlavičky nie sú
 - Blok obsahuje najviac jeden skok
 - Keď si bloky nakreslíme a dáme hrany že ako sa skáče, dostaneme graf toku riadenia
 - Generovanie kódu z dagu:
 - Zoberieme si blok. Ten má nejaké vstupné premenné a my chceme vypočítať výstupné premenné.
 - Vstupné premenné budú listy.
 - Keď máme príkaz $a = b + c$, tak nájdeme poslednú definíciu b, c v tom grafe čo kreslíme (čiže možno tie listy napríklad) a spravíme vrchol "+", ktorý bude mať synov b, c a napíšeme ku nemu "a".
 - Týmto vieme napríklad eliminovať spoločné podvyrazy, alebo mŕtvý kód
 - Poradie výpočtu musí byť podľa dagu
 - Polia, smerníky a volania procedúr (kvôli side-effectom) musia byť v rovnakom poradí, ako v neoptimalizovanom DAGu.
- **Generovanie kódu pokrývaním ozdobeného stromu.**
 - Máme AST. Budeme v ňom hľadať vzory a nahrádzať ich výsledkom (typicky registrom).
 - Každý vzor má zodpovedajúcu kachlicku (postupnosť instrukcií, ktorú ten vzor vygeneruje)
 - Každá kachlicka má svoju cenu, podľa zložitosti
 - Chceme pokryť strom kachlickami s čo najnižšou cenou
 - Veľké kachlicky sú dôsledkom zložitých instrukcií a adresných módov CISCu
 - Prístupy:

- Pazravo pokrývať vždy čo najväčšou kachlickou. Možno sa to potom neda dokončiť, keď zle začneš
- Kachlickovanie nahradiť syntaxou riadeným prekladom
 - Na vstupe je lineárny zápis medzijazyka
 - Budeme mať gramatiku pre nahradzovanie vzorov (čiže pravidla typu REGISTER -> VZOR)
 - Použitie pravidla vygeneruje kód
- Najlacnejšie pokrytie:
 - Pre všetky kachlicky pokrývajúce koreň ich skús a rekurzívne skús podstromy. Z nich vyskladaj minimum.
 - Treba memoizáciu zjavne

9. Optimalizácia kódu.

- **Optimalizácia základných blokov, optimalizačné transformácie.**
 - Použitie ďalšej informácie:
 - Premenná p.x z príkazu p má ďalšie použitie, ak v grafe toku riadenia existuje cesta z p do s, pričom s používa x a hodnota x sa na ceste nezmení.
 - Premenná, ktorá má ďalšie použitie je živá
 - Pomocou DAGu základného bloku z otázky 8 sa dá:
 - Eliminovať spoločné podvýrazy
 - Eliminovať mŕtvý kód
 - Keď vypočítame v DAGu pre blok B hodnotu premennej e a vieme, že e nemá ďalšie použitie (čiže to sa týka aj iných blokov), tak nemusíme tu e ani počítať a teda vďaka tomu možno nejakú časť toho dagu môžeme zmazať.
 - Priradenie do pola a smerníku zabíja
 - $x := a[i]$
 - $a[j] := y$ // Toto zabije vrchol vypočítaný v 1. kroku
 - $z := a[i]$ // A toto sa teda musí počítať z nových hodnôt
- **Eliminácia mŕtveho kódu, kukátková (peephole) optimalizácia, algebraické identity, redukcia v síle, vyhodnocovanie konštantných podvýrazov.**
 - Kukátková (peephole) optimalizácia:
 - Vidíme malú časť kódu, robíme lokálne optimalizácie:
 - Zbytočne LOAD a STORE po sebe
 - Loop unrolling
 - Loop fusion
 - Tok riadenia (skok na skok, nesúvisná podmienka)
 - Využívame ďalšie popísané body

- Algebraické identity:
 - $x + 0 = 0 + x = x$
 - $x / 1 = x$
 - Toto je pre kompilátor ľahké, napríklad ak vie, že niečo je 0
- Redukcia v sile:
 - $x^2 = x * x$
 - $2 * x = x + x$
 - $x / 2 = x * 0.5$
- Vyhodnocovanie konštantných podvyrazov:
 - Skladanie konštánt počas kompilácie
 - Napríklad v $2 * \pi * r$ sa vypočíta $2 * \pi$ počas kompilácie
- Komutatívnosť:
 - Keď máme jeden operand už v registroch, tak iba načítame druhý do druhého.
- **Analýza toku dát. Riešenie rovníc toku dát.**
 - Počas kompilácie chceme usudzovať o chovaní programu počas behu
 - Napríklad:
 - Invarianty:
 - Ak pri " $x = x + y$ " vieme, že y je vždy 1, tak INC je rýchlejšie
 - Ak pointer p vždy ukazuje do pola "a", tak netreba range check
 - Vlastnosti, ktoré platia často:
 - Prikaz $S1$ sa vykona častejšie ako $S2$, ...
 - Tu vieme optimalizovať rozdelenie premenných do registrov
 - Interminenty:
 - Premenná x niekedy nadobudne hodnotu rovnakú, ako y
 - Graf toku riadenia (blokový diagram)
 - Zaujímavé body tohto grafu:
 - Vstup do programu n_0
 - Body vetvenia (split)
 - $\text{succ}(n)$ = množina všetkých nasledovníkov
 - Body spájania (merge)
 - $\text{pred}(n)$ = množina všetkých predchodcov
 - Vystup z programu n_F
 - V zaujímavých bodoch chceme vedieť:
 - Z akeho príkazu priradenia pochádza hodnota nejakej premennej?
 - Ktorá premenná je už mŕtva?
 - Ake sú možné hodnoty premennej? (launch missiles)
 - Prenosové funkcie:
 - Charakterizujú účinok bloku B na informáciu o toku dát
 - Trieda prenosových funkcií:
 - Musí obsahovať identitu (prázdny blok)
 - Byť uzavretá na skladanie (lebo skladanie príkazov)

- Funkcie musia byt monotonne, cize pre vsetky funkcie f plati, ze $x \leq f(x)$, alebo pre vsetky plati $x \geq f(x)$
 - Toto je kvoli existencii pevneho bodu
- Dopredna analiza toku dat:
 - Ideme zhora dole
 - Pre kazdy uzol n je definovane:
 - in_n : informacia vchadzajuca do uzlu n
 - out_n
 - $f_{\{B_n\}}$: prenosova funkcia pre blok B_n
 - Riesenie musi splnat rovnice toku dat:
 - $in_{\{n_0\}} = I_0$ (cize asi nejaky hornik, alebo dolnik)
 - Pre vsetky n plati: $out_n = f_{\{B_n\}}(in_n)$
 - Pre vsetky n rozne od n_0 plati: $in_n = \text{zjednotenie } out\text{-ov vsetkych } z \text{ pred}(n)$
 - Mnozinu rovnici iterujeme, kym najdeme pevny bod
- Reverzna analiza toku dat:
 - Rovnice sa primerane uprava (napriklad ze out bloku musi byt zjednotenim in-ov successorov)
- Kompilator ma zostavit rovnice pre jednotlivy ulohy analyzy:
 - Reaching definitions (dopredna) -> ze definicia moze platit
 - Available expressions (dopredna) -> ze definicia musi platit
 - Live variables (reverzna)
- Metody riesenia rovnici:
 - Naivna iteracia:
 - Vzdy vypocitame vsetky rovnice a zistime, ci sa nieco zmenilo v tych in-och a out-och
 - Kazda rovnica je monotonna (to bol predpoklad), cize do konkretnej mnoziny in_k sa iba pridava, alebo iba odobera. Preto pevny bod existuje.
- Priradenie do premennej moze byt:
 - Zrejme
 - Skryte (napriklad cez smernik)
- Definicia "d" premennej "x" z bodu "p" moze platit (reaches) v bode q, ak existuje cesta z p do q, na ktorej sa nevyskytuje ziadne zrejme priradenie do "x".
 - Reaching definitions
 - Keby sa nemohlo vyskytovat ziadne (ani skryte) priradenie, tak by to bolo Available expressions
- V bloku S:
 - Nieco vznikne -> $gen[S]$
 - Nieco zanikne -> $kill[S]$
- Kazdy blok ma:
 - Zdedeny atribut $in[S]$
 - Syntetizovane atributy $out[S], gen[S], kill[S]$
 - $out[S] = gen[S] \cup (in[S] - kill[S])$

- Zive premenne:
 - Definujeme ten vzaz tak, ze:
 - kill je mnozina premennych v bloku, ktorym uzol priraduje hodnotu skor, ako ich pouzije
 - gen je mnozina premennych v bloku, ktore uzol pouzije skor, ako ich definuje